Guarded Cerebras Self-Consistency for Reliable, Compute-Aware In-Car Agents

Jivitesh Pasrija
FreudeDrive
jpasrija_be23@thapar.edu

Abstract

CAR-bench evaluates whether a tool-using in-car assistant stays consistently correct under ambiguity, missing capabilities, mutable state, multi-step tasks, and domain policy, crediting a task only when it succeeds in all three independent trials (Pass^3). Track 2 additionally caps compute: parallel inference is allowed, but every benchmark-visible assistant step may use at most five sequential LLM stages and average inference must stay below 500K tokens per task. We present *Guarded Cerebras Self-Consistency*, an agent that converts fast Cerebras inference into reliability without adding sequential latency on its common path. A zero-inference situation classifier launches one to three role-diverse gpt-oss candidates in parallel as a single sequential stage; canonical voting and deterministic checks over visible tool schemas, provenance, policy prerequisites, confirmation scope, and navigation state either accept a candidate immediately or trigger a bounded high-effort adjudication-and-repair path. A central gate enforces the five-stage depth limit and adapts breadth to a per-context token ledger, while all provider usage is aggregated into standard A2A `turn_metrics`. On a frozen 24-task, three-trial development panel the system reached 18/24 Pass^3 with exact accounting and zero infrastructure failures; a one-shot holdout reached 11/12 Pass^3.

1 Introduction

In-car assistants operate over interconnected tools and domain policies with a person in the loop, where an action that is right most of the time but wrong occasionally is worse than one that is reliably correct. CAR-bench targets this regime directly: it measures the *consistency* and *limit-awareness* of an LLM agent across multi-turn tasks involving incomplete information, absent capabilities, ambiguity, mutable vehicle state, and a simulated user [Kirmayr *et al.*, 2026]. A task is credited only when every required scoring component passes, and Pass^3 requires success in all three independent trials. This rewards agents that reliably manage interaction state rather than agents that occasionally sample a good answer.

Low-latency inference opens a natural lever for reliability: sampling several candidate next actions and reconciling them, in the spirit of self-consistency [Wang *et al.*, 2023]. Naive self-consistency, however, inflates both latency and token cost, which is precisely what Track 2 bounds. The track allows parallel inference within a step but limits each benchmark-visible assistant action to at most five *sequential* LLM stages and requires average inference below 500K tokens per task. The research question we address is therefore: *can fast parallel inference be turned into measurable reliability under a strict compute-awareness budget?*

Our answer is Guarded Cerebras Self-Consistency (SC). Its central idea is *selective concurrency*: a parallel wave of role-diverse candidates counts as one sequential stage, so reliability sampling adds provider work but not serial latency on the common path, and serial computation is spent only when deterministic evidence shows it can change the decision. Language models handle interpretation and plan generation; deterministic code owns validation, provenance, loop prevention, budget accounting, and protocol conformance in the style of reasoning-plus-acting agents [Yao *et al.*, 2023]. Concretely, this paper contributes:

- a compute-aware self-consistency architecture in which a parallel candidate wave is one sequential stage, decoupling reliability sampling from serial latency (Section 3);
- a deterministic verification layer for schemas, provenance, policy, confirmation scope, state, and failure loops that reads only A2A-visible information, making it invariant to renamed entities and reworded tasks;
- a central compute gate that enforces a hard five-stage sequential-depth limit, adapts breadth to a per-context token ledger, and aggregates all usage into standard A2A `turn_metrics`; and
- a disciplined empirical methodology—replay, negative controls, paired Pass^3 screens, and a one-shot holdout—yielding 18/24 development Pass^3, 11/12 holdout Pass^3, and exact accounting (Section 4).

2 Problem Setting

We model the agent as a partially observed transactional state machine. It receives only the A2A-visible policy, user messages, tool definitions, and tool results; it never executes

CAR-bench tools or reads evaluator state. Each next action must preserve outstanding intent, bind arguments to observed evidence, respect policy and confirmation state, and avoid claiming an unobserved mutation. Correct behavior thus requires the agent to represent intent and dependencies across turns, distinguish a *proposed* or *attempted* mutation from an *observed successful* one, use exact tool schemas and grounded identifiers, request clarification rather than guess among unresolved valid options, bind confirmation to the action actually presented, and preserve unsupported remainders instead of fabricating capability.

The Track 2 compute constraint reframes the design goal. Rather than deepening a single chain, we use Cerebras' low-latency inference to explore alternative next actions concurrently, and reserve sequential computation for cases where deterministic evidence indicates that another stage can change the outcome.

3 Compute-Aware Architecture

Each benchmark-visible assistant step follows the bounded pipeline of Figure 1: state reconstruction, an adaptive parallel proposal wave, canonical voting with deterministic verification, escalation only when needed, and cumulative accounting.

3.1 Adaptive Parallel Proposal Wave

A deterministic, zero-inference situation classifier labels the current visible turn for hallucination risk, disambiguation risk, policy risk, tool-error recovery, and finalization. Obvious final responses use a single candidate. Risky navigation, ambiguity, policy, confirmation, and multi-intent turns use up to three role-diverse candidates—policy-first, tool-first, and hallucination/disambiguation-aware—run concurrently through the direct Cerebras Python SDK over open-weight gpt-oss [Cerebras Systems, 2026]. Their wall-clock critical path is the maximum candidate duration, although accounting sums every call. Candidates emit *typed* next actions—a user response, tool calls with arguments, or both—rather than free prose, and are normalized before voting so that agreement concerns tool identity and arguments instead of lexical surface form. Clean agreement that passes every deterministic check exits after one sequential stage.

3.2 Deterministic Verification and State

The verifier rebuilds state from the conversation for each A2A `context_id`. It checks tool and parameter existence; required and conditional arguments; enums, numeric ranges, and additional properties; identifier provenance; policy prerequisites; ambiguity; confirmation requirements; and repeated failed calls. Reads returned as *explicitly unknown* are distinguished from state that was *never read*, which prevents blind writes without causing infinite re-reads. Preference retrieval is category and device scoped, so a value for one vehicle device is never applied to another. Confirmation binds to the immediately pending proposal and to naturally stated stable values; a changed recipient, target, operation, or material argument requires a fresh confirmation. Tool errors and exact failed-action signatures remain available to the next turn, and deterministic fallbacks are passed through the same verifier before crossing the A2A boundary. Because these rules use public schemas and visible results—not task identifiers or known solutions—they are invariant to renamed entities, reordered results, and changed task wording.

Table 1: Sequential-stage structure for one assistant step. A parallel wave is one stage; the fallback consumes none.

Depth	Stage	Trigger
1	1–3 parallel candidates	Always
+1	Resample	All candidates malformed
+1	Adjudicator	Conflict or guard finding
+1	Bounded retry	Concrete malformed output
+1	Repair	Adjudicated action blocked
0	Fallback	Budget exhausted

3.3 Escalation Under a Hard Depth Budget

When candidates disagree or a guard fires, one high-effort adjudicator sees the candidates and the structured findings. If the selected action still violates a guard, a bounded repair stage is permitted; malformed-output retries consume stages from the same budget. Once no stage remains, a safe deterministic fallback terminates the step. As summarized in Table 1, the budget is clamped to five sequential stages even if an environment variable requests more.

3.4 Accounting and Adaptive Resource Control

Each successful internal request contributes prompt, completion, thinking, cost, provider duration, quota wait, and call count to a per-context accumulator. Tool-call responses intentionally carry no metrics; their usage is flushed cumulatively on the next final user-facing response, matching the A2A evaluator contract. The reported standard fields `prompt_tokens`, `completion_tokens`, and `thinking_tokens` therefore include candidates, retries, adjudication, and repair. A conservative token ledger reduces breadth before exhaustion. Model, API base, service tier, reasoning effort, temperature, breadth, completion limits, and retry/backoff settings are environment-configurable, and the submitted runtime performs direct Cerebras-only inference with no secondary provider.

4 Evaluation

Methodology. We selected changes using deterministic unit and metamorphic tests, shadow replay of successful public trajectories, adversarial negative controls, fixed three-trial screens through the official Docker/A2A boundary, and paired task-level comparisons. We optimized complete Pass^3 flips rather than isolated trial gains, and paired comparisons used 10,000-sample bootstrap intervals. Full 24-task locked runs were performed only after a credible focused benefit. Regressing or uninformative changes were reverted; the final image is the strongest fully measured candidate, not an unmeasured union of experiments. The holdout was excluded from all development and run *once* after the candidate was frozen, without inspecting task details.

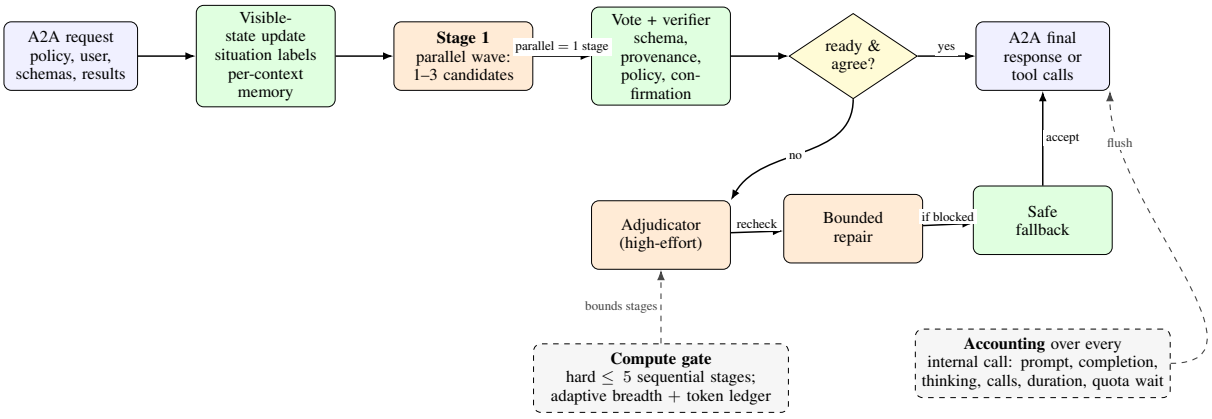

Figure 1: One benchmark-visible assistant step. Blue nodes are I/O; green nodes are deterministic and consume no LLM stage; orange nodes are LLM stages; the yellow diamond is a decision. The parallel candidate wave counts as a single sequential stage; adjudication and repair add stages only on conflict or a guard finding, under a central hard limit of five. A compute gate bounds sequential depth and sampling breadth, while usage over every internal call is accumulated and flushed through standard `turn_metrics` on the final response.

Table 2: Frozen system on the development panel and the one-shot holdout (three trials per task). Pass^3 credits a task only when all three trials pass.

Metric	Dev. panel	Holdout
Tasks × trials	24 × 3	12 × 3
Raw trial success	60/72	34/36
Pass^1	22/24	11/12
Pass^2	19/24	11/12
Pass^3	**18/24**	**11/12**
Base Pass^3	5/8	4/4
Hallucination Pass^3	8/8	3/4
Disambiguation Pass^3	5/8	4/4
Navigation/charging Pass^3	5/8	3/3
$\geq$6-action Pass^3	1/3	2/2
Mean total tokens	220,672	247,952
Mean A2A latency (s)	10.25	9.47
Max sequential depth	5	4
Infra/accounting failures	0	0

Results. Table 2 reports the frozen system on the development panel and the one-shot holdout. Perfect Hallucination Pass^3 on the development panel and strong holdout consistency (11/12) corroborate value from the capability, provenance, and confirmation checks, while both settings stayed well under the 500K average-token limit with a maximum sequential depth within budget.

Accounting integrity. The locked development run aggregated 14,176,597 prompt, 877,406 completion, and 834,371 thinking tokens (15,888,374 total) across 832 successful internal calls. All 72 per-trial token identities and cumulative A2A metric sums matched exactly. Sequential-depth observations at depths one through five were 221, 94, 22, 7, and 1. Mean summed provider duration (15.52 s) exceeded mean A2A wall time (10.25 s) because parallel candidate work overlaps in wall time—the quantitative signature of selective concurrency.

Offline and protocol audit. Offline verification comprised more than 300 deterministic unit tests plus 53 generated subtests. A replay audit covered 268 successful trajectories and 1,648 actions, with 9 of 9 adversarial negative controls detected. Docker checks verified linux/amd64, non-root execution, agent-card health, digest-pinned build stages, direct Cerebras-only inference, and the five-stage hard clamp. Source, image configuration, history, and layers were scanned for credentials and benchmark-specific answer material.

5 Discussion and Limitations

The architecture's main compute innovation is selective concurrency: extra samples add provider work but usually not serial latency, and serial adjudication is paid only when typed decisions or deterministic guards reveal a genuine conflict. The gap between mean A2A wall time and summed provider duration makes this overlap measurable. The disciplined revert record is itself a contribution: more rigid state certificates and wider high-risk sampling were each tested during development but blocked valid progress or failed to yield net Pass^3 gains, and were reverted rather than shipped without task-level evidence.

Remaining weaknesses are Base and Disambiguation consistency, long-horizon navigation/charging, and tasks requiring six or more actions; token and latency tails also exceed their medians, and the average-oriented 500K limit can be surpassed by individual long trajectories. Guard replay remains conservative, with a 5.70% block rate on successful public trajectories. We present these results as evidence of reliability, not as a claim about competition rank or hidden-set dominance: no public-panel result establishes hidden ranking. Future work should sharpen typed outstanding-intent and observed-success tracking while preserving the present fast path and avoiding false-positive blocking.

6 Conclusion

Guarded Cerebras Self-Consistency combines adaptive parallel reasoning with deterministic state, policy, provenance, and

confirmation checks. Its central depth and accounting gates make inference-time scaling auditable, while the fast path retains low sequential latency. On a frozen development panel and a one-shot holdout the frozen implementation delivered strong, consistently measured Pass^3 with exact resource accounting and no infrastructure failures, and is submission-ready as a public digest-pinned GHCR image.

References

[Cerebras Systems, 2026] Cerebras Systems. Cerebras cloud sdk. Software documentation, 2026.

[Kirmayr *et al.*, 2026] Johannes Kirmayr, Lukas Stappen, and Elisabeth André. CAR-bench: Evaluating the consistency and limit-awareness of LLM agents under real-world uncertainty. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 40599–40618, 2026.

[Wang *et al.*, 2023] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023.

[Yao *et al.*, 2023] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023.